

# Introduction to Message Passing Interface (MPI) 2026-2027

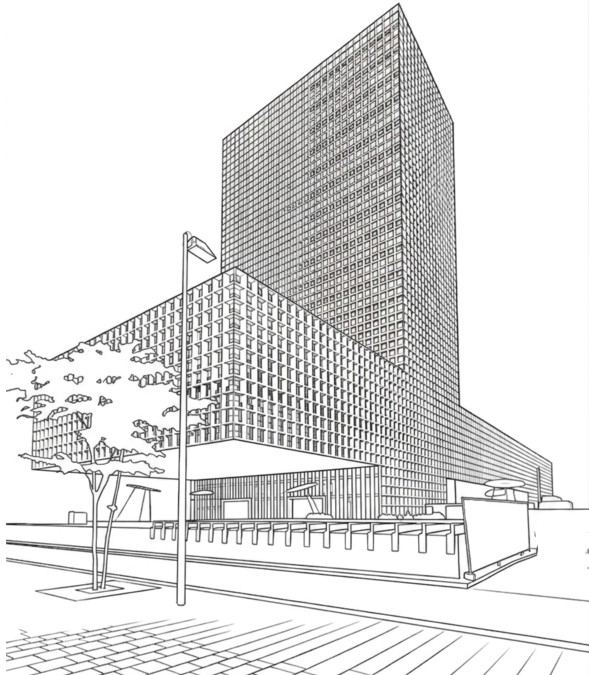
Lesson 2

Matteo Barborini

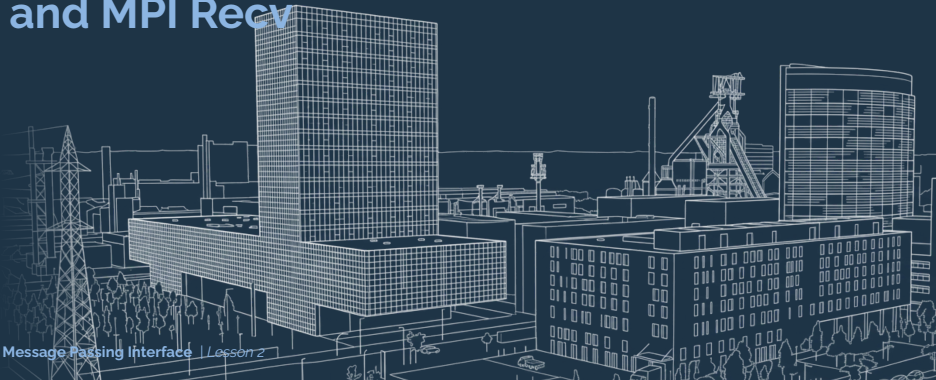
University of Luxembourg - HPC Platform

[matteo.barborini@uni.lu](mailto:matteo.barborini@uni.lu)

23rd of September 2026



# MPI Send and MPI Recv



# 1.3.MPI\_hello\_world\_send\_recv.c

## MPI Send and MPI Recv

```
1  #include <mpi.h>
2  #include <stdio.h>
3
4  int main(int argc, char *argv[]) {
5      int num_of_ranks, mpi_rank, mpi_rank_receive;
6
7      MPI_Init(&argc, &argv);
8      MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);
9      MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
10
11     if (mpi_rank != 0){
12         MPI_Send( &mpi_rank, 1, MPI_INT, 0, mpi_rank, MPI_COMM_WORLD);
13     } else {
14         printf("Hello from process %d of %d\n", mpi_rank, num_of_ranks);
15         for (int i = 1; i < num_of_ranks; i++) {
16             MPI_Recv( &mpi_rank_receive, 1, MPI_INT, i, i, MPI_COMM_WORLD, MPI_STATUS_IGNORE)
17             ;
18             printf("Hello from process %d of %d processes\n", mpi_rank_receive, num_of_ranks)
19             ;
20         }
21     }
22     MPI_Finalize();
23     return 0;
24 }
```

```
1 int MPI_Send(const void *buffer,  
2             int buffer_size,  
3             MPI_Datatype datatype,  
4             int destination_rank,  
5             int tag,  
6             MPI_Comm comm);
```

```
1 int MPI_Recv(void *buffer,  
2             int buffer_size,  
3             MPI_Datatype datatype,  
4             int source_rank,  
5             int tag,  
6             MPI_Comm comm,  
7             MPI_Status *status);
```

**buffer** is the address of the first element of the data to be sent.

**buffer\_size** is the number of elements to be sent. In our example we send only one integer, so the value is 1.

**datatype** is the type of the data to be sent. In our example **MPI\_INT** is used to send an integer.

**destination\_rank** is the rank of the process that will receive the data. The main rank receives the data in our example.

**source\_rank** is the rank of the process that will receive the data.

**tag** is an integer that can be used to identify the message. Important to match the send and receive operations.

**comm** is the communicator that defines the group of processes that can communicate with each other. In our case **MPI\_COMM\_WORLD** is used to communicate with all the processes.

**status** is the communicator that defines the group of processes that can communicate with each other.

| C Data Type        | MPI Datatype           |
|--------------------|------------------------|
| char               | MPI_CHAR               |
| signed char        | MPI_SIGNED_CHAR        |
| unsigned char      | MPI_UNSIGNED_CHAR      |
| short              | MPI_SHORT              |
| unsigned short     | MPI_UNSIGNED_SHORT     |
| int                | MPI_INT                |
| unsigned int       | MPI_UNSIGNED           |
| long               | MPI_LONG               |
| unsigned long      | MPI_UNSIGNED_LONG      |
| long long          | MPI_LONG_LONG          |
| unsigned long long | MPI_UNSIGNED_LONG_LONG |
| float              | MPI_FLOAT              |
| double             | MPI_DOUBLE             |
| long double        | MPI_LONG_DOUBLE        |

**MPI is a push first approach**, meaning that **the sender pushes the data to the receiver, and continues the execution without waiting, or knowing if the receiver has received the information.**

The receiver must be ready to receive the data, otherwise the data will be lost.

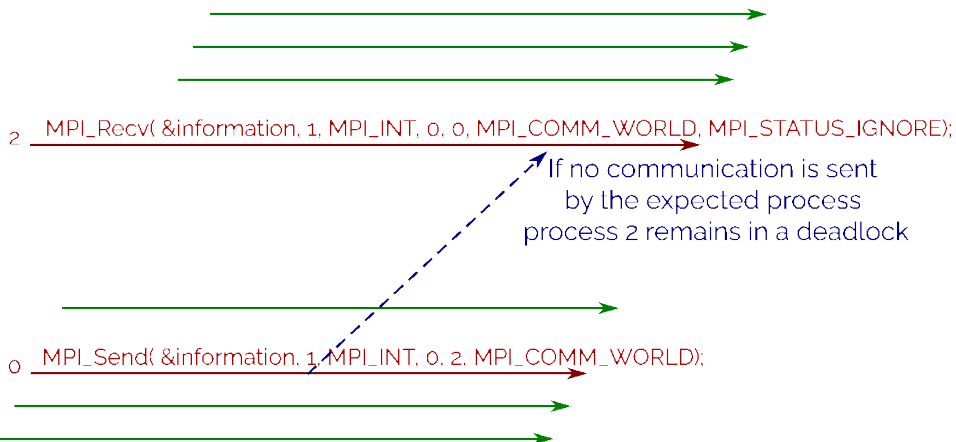
**The receiver does not know: the amount of data in the message, the sender of the message, the tag of the message.**

This information can be obtained by the receiver through the `MPI_Status` structure that contains at least 3 attributes

- `status.MPI_SOURCE` contains the rank of the sending process
- `status.MPI_TAG` contains the tag of the message assigned by the sender.
- `status.MPI_ERROR` contains the error code of the receive operation.

# Failed Transfer Creates a Deadlock

MPI Send and MPI Recv



The communication through MPI\_Send and MPI\_Recv functions

```
1 int MPI_Send(sending_buffer,  
2             sending_buffer_size,  
3             sending_datatype,  
4             destination_rank,  
5             sending_tag,  
6             sending_mpi_comm);
```

```
1 int MPI_Recv(receiving_buffer,  
2             receiving_buffer_size,  
3             receiving_datatype,  
4             source_rank,  
5             receiving_tag,  
6             receiving_mpi_comm,  
7             status);
```

is successful if the following conditions are met:

- `sending_tag = receiving_tag`
- `source_rank` and `destination_rank` must be compatible within each other in explicit communication.
- `receiving_mpi_comm = sending_mpi_comm`

Compatibility between sending and receiving buffers requires that the data type and size of the sending buffer is equal to that of the receiving buffer. A 'safe' communication can still happen if:

- `receiving_buffer_size ≥ sending_buffer_size`
- `receiving_datatype = sending_datatype`

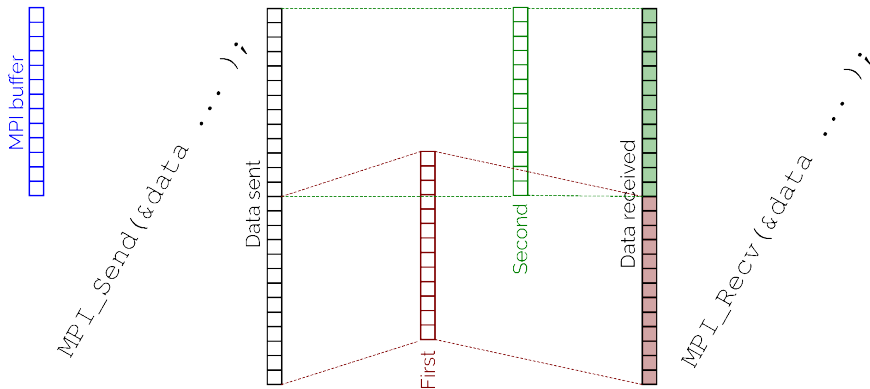
# MPI Send and MPI Recv Synchronizations

```
1 int MPI_Send(const void *data,  
2             int data_size,  
3             MPI_Datatype data_type,  
4             int destination_rank,  
5             int tag,  
6             MPI_Comm comm);
```

```
1 int MPI_Recv(void *data,  
2             int data_size,  
3             MPI_Datatype data_type,  
4             int source_rank,  
5             int tag,  
6             MPI_Comm comm,  
7             MPI_Status *status);
```

**MPI\_Send** is locally blocking, meaning that the routine MIGHT NOT RETURN until the memory holding the message is available to reuse. This depends on the difference between the information that must be sent and the buffer of the communications.

**MPI\_Recv** is blocking, meaning that the routine RETURNS only after the receive buffer contains the newly received message.



```

1 int MPI_Bsend(const void *data,
2             int data_size,
3             MPI_Datatype data_type,
4             int destination_rank,
5             int tag,
6             MPI_Comm comm);

```

```

1 // Legacy Signature
2 int MPI_Buffer_attach(void *buffer,
3                       int size);
4
5 // MPI_Count (MPI-4) Is used for
   datacounts exceeding 32-bit integer
   limits
6 int MPI_Buffer_attach_c(void *buffer,
7                         MPI_Count size);
8
9 // Safely return the buffer memory
10 int MPI_Buffer_detach(void *buffer,
11                      int *size)
12
13 int MPI_Buffer_detach_c(void *buffer,
14                        MPI_Count *size)

```

**MPI\_Bsend** is blocking, meaning that the routine will assume the message is fully contained in a buffer provided that the length is specified by the user through the function **MPI\_Buffer\_attach**.

**MPI\_Buffer\_attach** Links the MPI buffer to a predefined vector, with a fixed size. If **size** larger than a 32-bit integer use **MPI\_Buffer\_attach\_c**.

**MPI\_Buffer\_detach** (**MPI\_Buffer\_detach\_c**) returns the pointers to the buffer vector and buffer size memory.

# MPI\_Ssend (Synchronous) & MPI\_Rsend (Ready)

## MPI Send and MPI Recv Synchronizations

```
1 int MPI_Ssend(const void *data,  
2             int data_size,  
3             MPI_Datatype data_type,  
4             int destination_rank,  
5             int tag,  
6             MPI_Comm comm);
```

```
1 int MPI_Rsend(const void *data,  
2             int data_size,  
3             MPI_Datatype data_type,  
4             int destination_rank,  
5             int tag,  
6             MPI_Comm comm);
```

MPI\_Ssend has explicit blocking, similar to *standard* but it **will not return until the matching receive has been pointed**.

MPI\_Rsend **crashes if the receiver is not READY TO RECEIVE** the communication.

All subroutines start with the capital 'I' that stands for **I**nstantaneous

```
1 int MPI_I(,b,s,r)send(const void *data,  
2     int data_size,  
3     MPI_Datatype data_type,  
4     int destination_rank,  
5     int tag,  
6     MPI_Comm comm,  
7     MPI_Request *request);
```

```
1 // Wait until transfer done  
2 int MPI_Wait(MPI_Request *request,  
3     MPI_Status *status)  
4  
5 // Test transfer and proceed  
6 int MPI_Test(MPI_Request *request,  
7     int *flag,  
8     MPI_Status *status)
```

All `MPI_I(,b,s,r)send` **return immediately, thus the code continues.**

`MPI_Wait` waits for the send to be executed correctly (according to the type of send).

`MPI_Test` Test the specific requests, `flag` returns the status.

The `status` can be simply set in both cases to `MPI_STATUS_IGNORE`.

# Non-blocking Recv (MPI\_Irecv)

## MPI Send and MPI Recv Synchronizations

| Mode                | Function   | Key Mechanism                                       | Common Failures                                                                                          |
|---------------------|------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Standard</b>     | MPI_Send   | MPI chooses buffered                                | Deadlock if buffer is insufficient and receive is not executed.                                          |
| <b>Buffered</b>     | MPI_Bsend  | Uses explicit user defined MPI buffer.              | Buffer Overflow: Fails or crashes if attached buffer size is insufficient or missing MPI_BSEND_OVERHEAD. |
| <b>Synchronous</b>  | MPI_Ssend  | Completes only when matching MPI_Recv starts.       | Deadlock if receiving rank never calls MPI_Recv or if order is mismatched.                               |
| <b>Ready</b>        | MPI_Rsend  | Assumes receiver is already waiting.                | Crashes or silently drops data if matching MPI_Recv was not called beforehand.                           |
| <b>Non-blocking</b> | MPI_I*send | Initiates send asynchronously, returns MPI_Request. | Can modify data before MPI_Wait or MPI_Test completes (Data Race).                                       |

We can also define *non-blocking standard receive*

```
1 int MPI_Irecv(void *data,  
2             int data_size,  
3             MPI_Datatype data_type,  
4             int source_rank,  
5             int tag,  
6             MPI_Comm comm,  
7             MPI_Request *request);
```

```
1 int MPI_Recv(void *data,  
2             int data_size,  
3             MPI_Datatype data_type,  
4             int source_rank,  
5             int tag,  
6             MPI_Comm comm,  
7             MPI_Status *status);
```

The function again **returns immediately**, thus the code continues, and the receive operation must be verified through `MPI_Wait` or `MPI_Test`.

The functions `MPI_recv` and `MPI_Irecv` also accept the special variables `MPI_ANY_SOURCE` for the `source_rank` argument, which allows a process to receive a message from any source, and the variable `MPI_ANY_TAG` for the tag argument, thus accepting all sources.

When using non-blocking or blocking communications, it is possible to verify the status of the send operation through the instructions `MPI_probe` or `MPI_Iprobe`.

```
1 int MPI_Probe(int source_rank,  
2             int tag,  
3             MPI_Comm comm,  
4             MPI_Status *status);
```

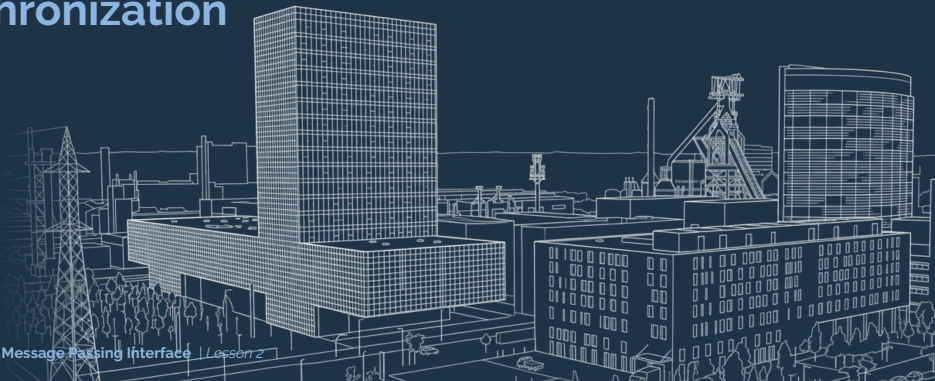
```
1 int MPI_Iprobe(int source_rank,  
2               int tag,  
3               MPI_Comm comm,  
4               int *flag,  
5               MPI_Status *status);
```

If in general the length of the data is unknown, it is possible to use the function `MPI_Get_count` to determine the number of elements received in a message.

```
1 int MPI_Get_count(const MPI_Status *status,  
2                  MPI_Datatype data_type,  
3                  int *count);
```

Here **count** is the number of received elements of the **data** communicated to the specific process and probed through `MPI_Probe` calls.

# MPI Synchronization



Blocks the caller until all processes in the communicator have called it.

```
1 int MPI_Barrier(MPI_Comm comm);  
2
```

It is used to synchronize processes at a specific execution point.

It ensures that all processes reach the same point and thus the same data state before proceeding.

No process can cross the barrier until **all** processes arrive.

```
1 // All processes execute their operations with different timings  
2  
3 . . .  
4  
5 // Barrier Ensure all processes finish before measuring time  
6 MPI_Barrier(MPI_COMM_WORLD);  
7
```

Overusing barriers causes idle processor time (waiting for the slowest process) and degrades overall performance.

Collective communications (e.g., **MPI\_Reduce**) already handle necessary synchronization internally.

# Exercise: Simple Dot product of two vectors



We recall that the dot product between two double precision vectors **a** and **b**, both of length  $N$  is defined as:

$$c = \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^N a_i b_i. \quad (1)$$

We define the simple serial code algorithm as

```
1  int num_of_elements=100;
2  double *a;
3  double *b;
4  double c = 0.0;
5  a = (double *)calloc(num_of_elements, sizeof(double));
6  b = (double *)calloc(num_of_elements, sizeof(double));
7  for (int i = 0; i < num_of_elements; i++)
8  {
9      a[i] = (double)i;
10     b[i] = (double)i;
11 }
12 for (int i = 0; i < num_of_elements; i++)
13 {
14     c+=a[i]*b[i];
15 }
16 printf(" %4.2f ", c);
```

Use as an example the code with MPI\_Send and MPI\_Recv for vectors introduces in this lesson.

1. Consider two vectors **a** and **b** of  $N$  elements each.
2. Rank 0 divides the vectors in  $M$  segments equal to the total number of MPI processes. (Warning!! One rank might hve less elements)
3. Rank 0 communicates the value of the lengths of each segment to the other ranks.
4. Rank 0 communicates the segments to each corresponding rank.
5. Each process multiplies the partial vectors. They do separate dot products.
6. Collect all values from each process to the main one, (rank 0)
7. Rank 0 sums the collected values and prints the integral.